28.1 记忆模型设计

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~300k tokens，输出 ~7k tokens（本节）

LLM 本身没有持久记忆——每次对话都是从零开始。用户昨天告诉 Agent 的偏好、上周讨论的项目决策、上个月记录的待办事项，在新对话中全部消失。OpenClaw 的记忆系统解决了这个问题：让 Agent 可以**跨会话记住和检索**重要信息。

本节分析记忆系统的设计哲学、文件布局和核心管理器。

28.1.1 "纯 Markdown 即记忆"的设计哲学

为什么不用数据库?

许多 AI Agent 框架将记忆存储在向量数据库（如 Pinecone、Weaviate）或关系型数据库中。OpenClaw 选择了一条不同的路径：**记忆的"源"是纯 Markdown 文件**。

```
~/.openclaw/
├── MEMORY.md              ← 长期记忆（手动维护或 Agent 写入）
├── memory/
│   ├── 2026-01-15.md      ← 日志式记忆（按日期自动生成）
│   ├── 2026-01-16.md
│   └── project-notes.md   ← 自定义记忆文件
```

这个设计有几个深层考量：

特性	Markdown 方案	数据库方案
可读性	用户可以直接阅读和编辑	需要专门的查询工具
版本控制	可以纳入 Git 管理	需要导出/迁移工具
Agent 写入	Agent 直接使用文件工具写入	需要专门的 API

特性	Markdown 方案	数据库方案
可移植性	复制文件即可迁移	需要数据库驱动/协议
调试	`cat MEMORY.md`	需要数据库客户端

换句话说，OpenClaw 把 Markdown 文件当作**真值源（Source of Truth）**，向量索引只是为了加速搜索而构建的**派生索引**。索引丢失可以重建，但 Markdown 文件包含了所有原始信息。

路径判定

`isMemoryPath` 函数定义了哪些文件属于"记忆"：

```
// src/memory/internal.ts

export function isMemoryPath(relPath: string): boolean {
  const normalized = normalizeRelPath(relPath);
  if (normalized === "MEMORY.md" || normalized === "memory.md") {
    return true;
  }
  return normalized.startsWith("memory/");
}
```

规则很简单：根目录下的 `MEMORY.md` （或 `memory.md` ），以及 `memory/` 目录下的所有 `.md` 文件。

28.1.2 记忆文件布局

长期记忆：MEMORY.md

`MEMORY.md` 是用户的**长期记忆文件**，通常包含持久的偏好、规则和知识。它可以被用户手动编辑，也可以被 Agent 在对话中写入。典型内容：

```
# 用户偏好
- 编程语言偏好：TypeScript、Python
- 编辑器：VS Code
- 沟通风格：简洁直接

# 项目约定
- 代码风格：使用 ESLint + Prettier
- 测试框架：Vitest
- 部署环境：AWS ECS

# 重要决策
- 2026-01-10：决定从 REST 迁移到 GraphQL
- 2026-01-15：选择 PostgreSQL 替代 MongoDB
```

日志记忆：memory/YYYY-MM-DD.md

`memory/` 目录下的日期命名文件是**会话记忆的快照**。当用户使用 `/new` 命令开始新会话时，`session-memory` 钩子会自动将当前会话的关键对话内容保存到这里：

```
// src/hooks/bundled/session-memory/handler.ts（简化）

const memoryDir = path.join(workspaceDir, "memory");
await fs.mkdir(memoryDir, { recursive: true });

// 生成带日期和描述的文件名
const filename = `${date}-${slug}.md`;  // 如 "2026-01-15-graphql-migration.md"

// 提取会话中的用户和助手消息（过滤掉工具调用）
const entry = messages
  .filter(msg => msg.role === "user" || msg.role === "assistant")
  .filter(msg => typeof msg.content === "string")
  .map(msg => `${msg.role}: ${msg.content}`)
  .join("\n\n");

await fs.writeFile(memoryFilePath, entry, "utf-8");
```

额外记忆路径

除了标准路径，用户可以通过配置添加额外的记忆目录：

```
// src/memory/internal.ts

export function normalizeExtraMemoryPaths(
  workspaceDir: string,
  extraPaths?: string[],
): string[] {
  return extraPaths
    ?.map(value => value.trim())
    .filter(Boolean)
    .map(value => path.isAbsolute(value)
      ? path.resolve(value)
      : path.resolve(workspaceDir, value))
    ?? [];
}
```

这允许用户将团队共享的知识库、项目文档等纳入记忆搜索范围。

文件发现

`listMemoryFiles` 函数扫描所有记忆文件：

```typescript
// src/memory/internal.ts（简化）

export async function listMemoryFiles(
  workspaceDir: string,
  extraPaths?: string[],
): Promise<string[]> {
  const result: string[] = [];

  // 1. 根目录下的 MEMORY.md 和 memory.md
  await addMarkdownFile(path.join(workspaceDir, "MEMORY.md"));
  await addMarkdownFile(path.join(workspaceDir, "memory.md"));

  // 2. memory/ 目录递归扫描
  await walkDir(path.join(workspaceDir, "memory"), result);

  // 3. 额外路径
  for (const inputPath of normalizeExtraMemoryPaths(workspaceDir, extraPaths)) {
    const stat = await fs.lstat(inputPath);
    if (stat.isDirectory()) await walkDir(inputPath, result);
    else if (stat.isFile() && inputPath.endsWith(".md")) result.push(inputPath);
  }

  // 4. 去重（通过 realpath）
  return dedupeByRealpath(result);
}
```

注意安全措施： `walkDir` 会**跳过符号链接**（`entry.isSymbolicLink()`），防止通过符号链接引入不可控的文件。

28.1.3 记忆管理器（`src/memory/manager.ts`）

MemoryIndexManager

`MemoryIndexManager` 是记忆系统的核心引擎。它负责：文件索引、向量嵌入、BM25 全文搜索、混合排序、缓存管理。

```typescript
// src/memory/manager.ts

export class MemoryIndexManager implements MemorySearchManager {
  private readonly cfg: OpenClawConfig;
  private readonly agentId: string;
  private readonly workspaceDir: string;
  private readonly settings: ResolvedMemorySearchConfig;
  private provider: EmbeddingProvider;   // 嵌入向量提供者
  private db: DatabaseSync;              // SQLite 数据库
  private watcher: FSWatcher | null;     // 文件监视器
  private dirty: boolean;               // 是否需要重新索引
  // ...
}
```

单例缓存

`MemoryIndexManager` 通过静态方法 `get` 获取实例，使用全局缓存避免重复创建：

```typescript
// src/memory/manager.ts

const INDEX_CACHE = new Map<string, MemoryIndexManager>();

static async get(params: {
  cfg: OpenClawConfig;
  agentId: string;
}): Promise<MemoryIndexManager | null> {
  const settings = resolveMemorySearchConfig(cfg, agentId);
  if (!settings) return null;

  // 缓存键 = agentId + workspaceDir + 配置序列化
  const key = `${agentId}:${workspaceDir}:${JSON.stringify(settings)}`;
  const existing = INDEX_CACHE.get(key);
  if (existing) return existing;

  // 创建嵌入提供者
  const providerResult = await createEmbeddingProvider({ ... });

  // 创建并缓存管理器
  const manager = new MemoryIndexManager({ ... });
  INDEX_CACHE.set(key, manager);
  return manager;
}
```

> **衍生解释：单例模式与缓存键**
>
> 单例模式（Singleton Pattern）确保一个类只有一个实例。这里使用 Map 作为缓存，Key 由 agentId + 配置序列化组成。这意味着同一个 Agent 使用相同配置时共享同一个管理器实例，但不同 Agent 或不同配置会创建独立实例。这比传统的单例更灵活——它是**参数化单例**。

配置解析

记忆搜索的配置由 `resolveMemorySearchConfig` 统一解析，支持全局默认值和每个 Agent 的覆盖：

```typescript
// src/agents/memory-search.ts（简化）

export type ResolvedMemorySearchConfig = {
  enabled: boolean;
  sources: Array<"memory" | "sessions">;      // 数据源
  extraPaths: string[];                        // 额外路径
  provider: "openai" | "local" | "gemini" | "voyage" | "auto";  // 嵌入提供者
  model: string;                               // 嵌入模型
  store: {
    driver: "sqlite";
    path: string;                              // 索引文件路径
    vector: { enabled: boolean; extensionPath?: string };
  };
  chunking: { tokens: number; overlap: number };  // 分块参数
  query: {
    maxResults: number;                        // 最大返回结果数
    minScore: number;                          // 最低分数阈值
    hybrid: {
      enabled: boolean;                        // 是否启用混合搜索
      vectorWeight: number;                    // 向量权重（默认 0.7）
      textWeight: number;                      // 文本权重（默认 0.3）
      candidateMultiplier: number;             // 候选倍数
    };
  };
  cache: { enabled: boolean; maxEntries?: number };
  // ...
};
```

默认值经过精心调校：

参数	默认值	含义
`chunking.tokens`	400	每个分块约 400 token（~1600 字符）
`chunking.overlap`	80	相邻分块重叠 80 token
`query.maxResults`	6	最多返回 6 条结果
`query.minScore`	0.35	低于 0.35 分的结果被丢弃
`hybrid.vectorWeight`	0.7	向量相似度占 70% 权重
`hybrid.textWeight`	0.3	关键词匹配占 30% 权重

文件分块

Markdown 文件被分割成固定大小的**分块（Chunk）**，每个分块独立嵌入向量：

```
// src/memory/internal.ts

export function chunkMarkdown(
  content: string,
  chunking: { tokens: number; overlap: number },
): MemoryChunk[] {
  const maxChars = Math.max(32, chunking.tokens * 4); // 粗估：1 token ≈ 4 字符
  const overlapChars = Math.max(0, chunking.overlap * 4);
  const chunks: MemoryChunk[] = [];

  let current: Array<{ line: string; lineNo: number }> = [];
  let currentChars = 0;

  for (let i = 0; i < lines.length; i++) {
    const lineSize = line.length + 1;
    if (currentChars + lineSize > maxChars && current.length > 0) {
      flush();          // 输出当前分块
      carryOverlap();   // 保留尾部作为下一分块的开头
    }
    current.push({ line, lineNo: i + 1 });
    currentChars += lineSize;
  }
  flush();
  return chunks;
}
```

> **衍生解释：分块与重叠（Chunking & Overlap）**
>
> 为什么要分块？因为嵌入模型有输入长度限制（如 8192 token），而且更短的文本通常能产生更精确的语义向量。分块的挑战在于**语义断裂**——一段连贯的论述可能被截断在两个分块的边界。**重叠（Overlap）** 通过让相邻分块共享一部分文本来缓解这个问题。例如 80 token 的重叠意味着分块 A 的最后 80 个 token 也会出现在分块 B 的开头。

文件监视与脏标记

管理器使用 `chokidar` 监视记忆文件的变更，并设置 `dirty` 标记：

```
// src/memory/manager.ts（简化）

private ensureWatcher() {
  const watchPaths = [
    path.join(this.workspaceDir, "MEMORY.md"),
    path.join(this.workspaceDir, "memory.md"),
    path.join(this.workspaceDir, "memory"),
  ];

  this.watcher = chokidar.watch(watchPaths, {
    ignoreInitial: true,
    awaitWriteFinish: { stabilityThreshold: 1500 },
  });

  const markDirty = () => {
    this.dirty = true;
    this.scheduleWatchSync();  // 防抖后触发重新索引
  };

  this.watcher.on("add", markDirty);
  this.watcher.on("change", markDirty);
  this.watcher.on("unlink", markDirty);
}
```

当 Agent 调用 `memory_search` 时，如果 `dirty` 为 `true`，会先触发增量同步再执行搜索——实现**懒索引（Lazy Indexing）**。

本节小结

1. **"Markdown 即记忆"** 是核心设计哲学——纯文本文件作为真值源，向量索引只是派生数据。
2. **记忆有两层结构**—— `MEMORY.md` 存储长期知识， `memory/YYYY-MM-DD.md` 存储会话快照。
3. **MemoryIndexManager 是核心引擎**——通过参数化单例缓存、文件监视、懒索引等机制管理整个记忆生命周期。
4. **分块算法以行为单位**——按字符数估算 token 数，支持重叠以避免语义断裂。
5. **文件监视实现实时感知**—— `chokidar` 监听文件变更，设置脏标记，搜索时按需重建索引。
6. **配置支持多层覆盖**——全局默认值 → Agent 级别覆盖，包括嵌入提供者、分块策略、搜索参数等。